STEP 1 — Open Google Colab

STEP 2 — Import Required Libraries

```
# ================================
# Data handling libraries
# ================================
import numpy as np                      # For numerical operations
import pandas as pd                     # For handling tabular data

# ================================
# Hugging Face libraries
# ================================
from datasets import load_dataset       # To load dataset easily
from transformers import (
    AutoTokenizer,                      # For tokenization
    AutoModelForSequenceClassification, # Pre-trained model
    Trainer,                            # Training API
    TrainingArguments                   # Training configuration
)

# ================================
# Evaluation libraries
# ================================
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusio
import matplotlib.pyplot as plt
import seaborn as sns

# ================================
# PyTorch
# ================================
import torch
```

STEP 3 — Load and Explore Dataset

```
# Load IMDb dataset
dataset = load_dataset("imdb")

# Display dataset structure
print(dataset)

# Show sample record
print(dataset["train"][0])
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https:/
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public mode
  warnings.warn(
README.md:       7.81k/? [00:00<00:00, 352kB/s]
plain_text/train-00000-of-                                    21.0M/21.0M [00:00<00:00, 26.0MB/s]
00001.parquet: 100%
plain_text/test-00000-of-                                     20.5M/20.5M [00:00<00:00, 51.6MB/s]
00001.parquet: 100%
plain_text/unsupervised-00000-of-                             42.0M/42.0M [00:00<00:00, 20.2MB/s]
00001.p(…): 100%
Generating train split: 100%                    25000/25000 [00:00<00:00, 80414.50 examples/s]
Generating test split: 100%                     25000/25000 [00:00<00:00, 102742.88 examples/s]
Generating unsupervised split: 100%             50000/50000 [00:00<00:00, 109825.06 examples/s]
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label'],
        num_rows: 50000
    })
})
{'text': 'I rented I AM CURIOUS-YELLOW from my video store because of all the controver
```

```
# Check class distribution
train_labels = dataset["train"]["label"]

# Count values
unique, counts = np.unique(train_labels, return_counts=True)
print(dict(zip(unique, counts)))
```

```
{np.int64(0): np.int64(12500), np.int64(1): np.int64(12500)}
```

STEP 4 — Tokenization & Preprocessing

```
# Load pre-trained BERT tokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
def tokenize_function(example):
    return tokenizer(
```

```
        example["text"],
        padding="max_length",     # Pad to fixed length
        truncation=True,          # Truncate long text
        max_length=128            # Sequence length
    )

# Apply tokenizer
tokenized_dataset = dataset.map(tokenize_function, batched=True)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests t
```

config.json: 100% 570/570 [00:00<00:00, 36.2kB/s]

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 4.69kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 5.95MB/s]

tokenizer.json: 100% 466k/466k [00:00<00:00, 8.48MB/s]

Map: 100% 25000/25000 [00:26<00:00, 973.91 examples/s]

Map: 100% 25000/25000 [00:25<00:00, 1055.70 examples/s]

Map: 100% 50000/50000 [00:53<00:00, 892.00 examples/s]

STEP 5 — Prepare Dataset for Training

```
# Rename label column to 'labels' (required by Trainer)
tokenized_dataset = tokenized_dataset.rename_column("label", "labels")

# Set format for PyTorch
tokenized_dataset.set_format("torch", columns=["input_ids", "attention_mask", "labels
```

STEP 6 — Train–Test Split

```
train_dataset = tokenized_dataset["train"]
test_dataset = tokenized_dataset["test"]
```

STEP 7 — Load Pre-trained Model

```
model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2   # Positive & Negative
)
```

```
model.safetensors: 100%                                          440M/440M [00:04<00:00, 183MB/s]

Loading weights: 100%      199/199 [00:00<00:00, 327.69it/s, Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                          | Status     |
---------------------------------------------+------------+-
cls.predictions.bias                         | UNEXPECTED |
cls.predictions.transform.dense.weight       | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight   | UNEXPECTED |
cls.seq_relationship.bias                    | UNEXPECTED |
cls.seq_relationship.weight                  | UNEXPECTED |
cls.predictions.transform.dense.bias         | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias     | UNEXPECTED |
classifier.weight                            | MISSING    |
classifier.bias                              | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok
- MISSING   :those params were newly initialized because missing from the checkpoint. (
```

STEP 8 — Define Training Configuration

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)

    precision, recall, f1, _ = precision_recall_fscore_support(labels, predictions, a
    acc = accuracy_score(labels, predictions)

    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }
```

```
training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_dir="./logs",
)
```

```
`logging_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD_LOGGIN
```

STEP 9 — Train the Model

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics,
)
trainer.train()
trainer.evaluate()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning
  super().__init__(loader)
```

STEP 10 — Model Evaluation

```
predictions = trainer.predict(test_dataset)
preds = np.argmax(predictions.predictions, axis=-1)
labels = predictions.label_ids
cm = confusion_matrix(labels, preds)

plt.figure()
sns.heatmap(cm, annot=True, fmt="d")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```